调用内部：消息、流与工具调用

讲座 07 · 2026 年 9 月 22 日

layout: default

今天

三个部分

1. 一次 POST，一次回复
2. 流
3. 工具，以及线路上的循环

分析从这里开始。前三周没有作业分发。

layout: default

实验课 01，三周前

我们从未打开的卡片

<ph-stack-bold class="text-3xl text-blue-600 mb-3" />
组装
L02 、L03、仓库地图
<ph-chat-text-bold class="text-3xl text-amber-600 mb-3" />
回答
一行配置
<ph-pencil-simple-bold class="text-3xl text-blue-600 mb-3" />
应用
F5 、F6 、diff、提交

layout: default

为什么是现在

三个日期，一次调用

	时间	需要什么	
	本周	你编写客户端。`ENDPOINT.md`，F1	
	周四晚上	`taskr` 中一个 AI 功能，两小时	
	第 5 周	同样的调用，加上工具，无人值守	

layout: section

一次 POST

请求，逐字段

layout: default

ENDPOINT.md

请求对象

```
POST {base_url}/chat/completions

{
  "model": "qwen3.5:9b",
  "messages": [
    {"role": "system",    "content": "..."},
    {"role": "user",      "content": "..."},
    {"role": "assistant", "content": "..."},
    {"role": "user",      "content": "..."}
  ],
  "stream": true
}
```

这是你的程序与模型之间的完整接口

layout: statement

没有 **session 字段**。没有地方可以 *放置* 记忆。

layout: default

ENDPOINT.md，评分标准

URL 规则

	配置的 `base_url`	请求发往哪里	
	`http://localhost:11434/v1`	`…:11434/v1/chat/completions`	
	`http://localhost:11434/v1/`	`…:11434/v1/chat/completions`	
	`https://api.example.edu`	`…example.edu/chat/completions`	

去掉一个尾部斜杠，附加路径。不要凭空发明 `/v1`。

layout: default

不在最小值中

另外三个请求字段

- `temperature` — L02 的采样步骤，带一个刻度盘
- `max_tokens` — 一个上限，也是测试的最佳朋友
- `tools` — 今天的后半部分

省略 `temperature`，服务器会自己选。它选的不是零。

layout: default

兼容性

"兼容"承诺什么

- Ollama、llama.cpp、vLLM、每个供应商：相同的对象
- 没有人批准它。一个供应商的 API 赢了
- 在推理、工具、`usage` 处 fray

`ENDPOINT.md` 画出了界限：文本聊天补全，仅此而已。

layout: section

一次回复，三个结尾

回来的是什么，以及它强制什么

layout: default

实时 · stream false

回复信封


```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "HTTP 状态 404 意味着请求的资源在服务器上未找到。",
      "reasoning": "思考过程：\n\n1.  **分析请求：** ..."
    },
    "finish_reason": "stop"
  }],
  "usage": {"prompt_tokens": 31, "completion_tokens": 465, "total_tokens": 496}
}
```


实时运行。一个简短的问题，一个简短的答案。

layout: default

每个回复都有一个结束原因

三个结束原因

	值	你的程序的义务	
	`stop`	模型结束了它。F5 可能会寻找编辑块	
	`length`	被截断了。一个明确的错误，永远不是编辑	
	`tool_calls`	它在请求。后半部分	

给你的代码的信息，不是给用户的。

layout: default

实时 · max_tokens 8

一个被截断的回复

```
"message": {
  "role": "assistant",
  "content": "",
  "reasoning": "Here's a thinking process that leads to"
},
"finish_reason": "length"
```

空内容。模型不是什么都没说。

layout: default

usage

你的估算器评分的依据

```
"usage": {"prompt_tokens": 31, "completion_tokens": 465, "total_tokens": 496}
```

- `/tokens` 预测。这个字段计数了
- 发送一次，比较，校准。一个下午

F1 允许估算粗略。它不允许它未校准。

layout: default

465 计费，14 可见

推理在哪里到达

	服务器	思考落在哪里	
	Ollama `/v1`	`content` 旁边的一个 `reasoning` 字段	
	其他	在 `content` 内部，在 `<think>` 标签之间	

三个决定：显示它吗？保留在历史中吗？其中的编辑块是一个提议吗？

layout: section

流

`stream: true`，第 1 阶段始终如此

layout: default

实时 · 同样的问题

一个回复作为事件

```
data: {"choices":[{"delta":{"role":"assistant","content":"","reasoning":"Thinking"}}]}
data: {"choices":[{"delta":{"content":"","reasoning":" Process"}}]}
...

data: {"choices":[{"delta":{"content":" server"}}]}
data: {"choices":[{"delta":{},"finish_reason":"stop"}]}
data: [DONE]
```

一句话，516 个事件。大多数根本不携带文本。

layout: default

你的客户端无怨无悔地接受什么

六种事件

	事件	携带	
	开头增量	`role` 。这里，还有一个推理片段	
	仅推理增量	思考。数百个	
	内容增量	用户看到的文本	
	空增量	`finish_reason`	
	仅 usage 事件	根本没有 `choices`	
	`[DONE]`	不是 JSON。正常结束	

layout: statement

在事件 400 处死掉的流已经 **打印了大部分回复**。

layout: default

F1，以及每个动词的原因

失败流规则

	你的程序	因为	
	不记录助手回合	下一个请求会携带一个从未完成的回复	
	不把任何东西当作编辑	F5 和 F6 依赖于提议是完整的	
	说出来，保持可用	用户必须能够重试	

具有安全后果的合同条款。

layout: default

实验课 00 的 bin/check 中有一行关于这个

没有人报告的截断

- Ollama 保持 4k tokens，除非被告知
- 超过它：没有错误，没有 `length`，没有消息
- 它丢弃 **开头**。你的系统提示，你的第一个文件

把它提高到 32k。你 24000 的预算为回复留有余地。

layout: default

与邻居五分钟

你没有看到的 451 个 token

那个回复计费了 **465** 个 completion token。**14** 个是那句话。
你的程序将该回合附加到历史并计费到 `context_budget`。

写一行：`charged: <数字>, reasoning: keep | drop`

然后准备好说你的选择在下一个请求上花费了什么。

layout: section

工具信封

同样的四个部分，在线路上

layout: default

L06：一个名称，一个描述，一个模式

动作的四个部分

部分	谁的
名称和描述	模型的。它读取来选择
参数契约	你的。检查请求的代码
分发	你的。做这件事的代码
观察	你的。列表中的下一条消息

描述一个工具给模型提供了请求的能力。仅此而已。

layout: default

实时 · 三个工具

一个工具定义

```
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "从仓库读取一个 UTF-8 文本文件。",
    "parameters": {
      "type": "object",
      "properties": {"path": {"type": "string"}},
      "required": ["path"]
    }
  }
}
```

分析规范命名的三个：`read_file`、`write_file`、`run_command`。

layout: default

测量，同样的三条消息

一个工具定义花费什么

	请求	`prompt_tokens`	
	没有 `tools`	71	
	有三个工具	471	

每个请求 400 个 token，在运行的其余部分。

layout: default

实时 · 模型的回答

回复是一个请求

```
"message": {
  "role": "assistant",
  "content": "",
  "tool_calls": [{
    "id": "call_kk9dbt2h",
    "type": "function",
    "function": {"name": "read_file", "arguments": "{\"path\":\"taskr/cli.py\"}"}
  }]
},
"finish_reason": "tool_calls"
```

它没有回答。它请求了。

layout: default

这些每一个只咬一次

那个回复中的四件事

	字段	它花费你什么	
	`content: ""`	空不是完成。分支在 `finish_reason` 上	
	`arguments`	模型写的一个 **字符串** JSON	
	`id`	结果如何回来。保留它	
	`tool_calls`	一个列表。它可能一次请求三个	

layout: default

两个附加，然后发送

往返

```
{"role": "assistant", "content": "", "tool_calls": [ ...exactly as received... ]},
{"role": "tool", "tool_call_id": "call_kk9dbt2h", "content": "\"\"\"Argument parsing and command dispatch for the taskr CLI.\"\"\"\n..."}
```

第一行是每个人都忘记的那一行。

layout: default

周四，选一个

原生调用或文本协议

	原生 `tools`	JSON 在散文中	
	一个角色，一个 id，一个服务器看到的模式	适用于任何模型	
	一些服务器上片段化的参数	你写解析器	
	第 5 周要求	围栏 JSON，周围有散文	

第 2、3 和 4 部分是相同的代码，无论哪种方式。

layout: section

线路上的循环

L06 的六行，真实运行

layout: default

L06，周四

讲座中的循环

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text
    for call in reply.tool_calls:
        result = execute(call)
        messages.append(result)
```

与周四相同。你现在知道每一行发送什么。

layout: default

逐行

同样的六行，作为请求

	行	在线路上	
	`model.call(..., tools=)`	POST，加上 400 个 token	
	`if not reply.tool_calls`	在 `finish_reason` 上分支	
	`execute(call)`	第 2 和第 3 部分。稍后：审批	
	`messages.append(result)`	两个附加，不是一个	

layout: default

一次运行，已捕获

在你自己的仓库上的运行

	#	工具	参数	tokens	
	1–3	`read_file`	`cli.py`、`task.py`、`store.py`	471 → 1214	
	4–6	`write_file`	`task.py`、`store.py`、`cli.py`	1638 → 2454	
	7	`run_command`	`python -c "from cli import main; …"`	3097	
	8–9	`run_command`	`rm -f ~/.taskr.json && …`	3168 → 3252	

没有执行任何东西。每个 `run_command` 被告知 `exit 0`。

layout: default

跟踪的大部分

它做对了什么

- 读取了 `cli.py`，然后是 `store.py`，然后是 `task.py`
- 正确的三个文件。一个标志触及所有它们
- 按依赖顺序写入，不是读取顺序

一个 9B 在写入之前读取，并按顺序序列化了它的写入。

layout: statement

`rm -f ~/.taskr.json` — 而且 *git 从未知道那个文件存在。*

layout: default

最后一列

它选择添加的上下文

	步骤	`prompt_tokens`	
	1	471	
	4	1638	
	9	3252	

九步中七倍，而且它从未说它完成了。

layout: default

L05：找到决策，不是罪魁祸首

实际失败的东西

	什么	修复	
	站在 `rm -f` 和 `$HOME` 之间	代码中的审批	
	识别完成	六个命名的停止条件	
	使命令的结果真实	一个真正运行的 `run_command`	
	限制增长的上下文	一个 token 预算，而不仅仅是一个步骤限制	

模型仔细阅读。缺失的是工具链。

layout: default

今晚

`ENDPOINT.md` 作为一个清单

- URL 规则，条件认证，请求对象
- 六种事件类型，三个结束原因，推理
- 失败流规则

每一个都是你可以针对一个假流编写的测试。不需要端点。

layout: statement

将调用做成一个带默认值的 *参数*。`def plan(task, call=complete)`

layout: default

日期

未来一周

	时间	什么	
	今天，在此之后	今晚的简报	
	今晚，7 到 9 PM	黑客松 1，Leinweber 1355	
	10 月 6 日，周二	构建到期。分析发布	

10 月 6 日之前没有任何关于分析的东西被分发。

layout: default

带走这些

三个问题

1. 流在四十个词后死亡。`messages` 里有什么？
2. 三个工具花费 400 个 token。多少个时 24000 会崩溃？
3. 它未读地写了 `store.py` 的约定。散文还是代码？

layout: default

即将到来

六行缺少什么

- 六个运行结束的原因，每个在日志中命名
- 放在你未读动作前面的提示

接下来：今晚的简报。

layout: end